AgentCore as the Sandbox in Agentic RL

AgentCore Runtime as the execution sandbox, integrated with slime

Kaige Yang — Senior Applied Scientist, AWS

September 10, 2026

Outline

1. 1. Why sandboxes — what frontier labs run in them
2. 2. Agentic RL architecture & the training pipeline
3. 3. Foundations of AgentCore Runtime
4. 4. Integrating AgentCore into slime as the sandbox
5. 5. The demos — one storyline
6. 6. Cross-cutting mechanisms
7. References

Section 1

1. Why sandboxes — what frontier labs run in them

Purpose & roadmap — AgentCore as the sandbox for agentic RL

One idea: in agentic RL the policy acts over many turns against an executable environment, and a **programmatic checker** — not an LLM judge — turns the outcome into a scalar reward (RLVR).

Roadmap (6 sections):

1. **Why sandboxes** — what frontier labs run in sandboxes, and how they score it.
2. **Architecture & pipeline** — reference architecture, the five components, slime + its async rollout.
3. **Foundations of AgentCore** — Runtime / session / microVM, CPU vs GPU compute, session lifecycle & launch/terminate.
4. **Integrating into slime** — the call path, one hook, the episode loop, and the key code.
5. **The demos** — all 9 at a glance, then **one slide per demo** (config, checker, measured eval/latency, honest status, real use case).

Purpose & roadmap — AgentCore as the sandbox for agentic RL (cont.)

6. **Cross-cutting mechanisms** — episode shape, TITO, the scaling `min()` law, one-episode→many-samples, and the resilience levers.

*Scope: AgentCore = the execution **sandbox**; **slime** = the trainer; the **harness stays in-process**.*

Executive summary — the one slide

- **Problem.** Agentic RL runs **untrusted model output** in an isolated environment thousands of times per run and scores it with a **programmatic checker** — the *sandbox* layer, not the GPU, is often the bottleneck (Kimi K2 [5]: **>10,000 concurrent sandboxes**).
- **Solution. AgentCore Runtime is that sandbox** — one isolated microVM session per episode, **in your own AWS account** (IAM/SigV4, VPC), bring-your-own container, CPU by default and **GPU on Instances** — wired into the **slime** trainer through **one hook**, harness in-process.
- **Proven.** The full loop runs **end-to-end on real hardware** across **7 main demos + 1 experimental** (code, math, SQL, RAG, tool use, data science, terminal tasks, CUDA kernels), each smoke a standard **10-step run** with real rewards from an in-sandbox checker. **Most literature tasks need no image**: they train on the default Code Interpreter.
- **How you adopt it.** Start on the **default Code Interpreter** (no image) → seed a task → train via one `generate` hook; build your own Runtime only when the task needs a custom image, a > 2 GB toolchain or a GPU.

What tasks do frontier labs run in sandboxes

Untrusted output runs in an isolated sandbox; a **programmatic verifier** produces the reward. This is the dominant recipe in recent tech reports — Kimi K2 [5] reports **>10,000 concurrent sandboxes** for agentic tool-use / code RL.

Two verifier styles: *verifiable / rule-based* (binary, ungameable) vs *reference-based judge* (open-ended). This project uses **RLVR only**.

Model	Verifier	Tasks	Reward
DeepSeek-R1 [2]	rule verifier + compiler	math, competitive code	tests pass / match
Qwen3 [3]	RLVR + live tool env	math, code, tool-use, RAG	rule + judge
Llama-Nemotron [4]	LLM-judge	science, math, instruction	match + format
Kimi K2 [5]	simulator + real exec sandboxes	agentic tool use, code, SWE	tests + checks

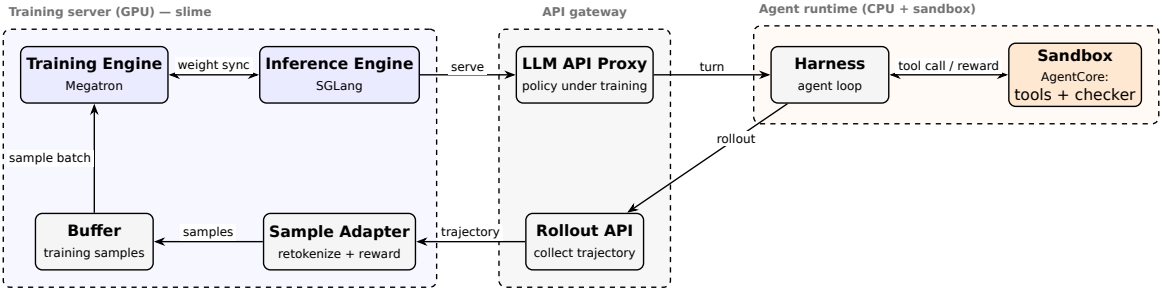
What tasks do frontier labs run in sandboxes (cont.)

Same recipe throughout this deck: ***one isolated sandbox per episode****, a* ***programmatic checker*** *for the reward.*

Section 2

2. Agentic RL architecture & the training pipeline

The reference architecture — one diagram



Generation lane (top, →); training-data lane (bottom, ←); disaggregated design after Agent Lightning [1], slime [9] as trainer. ***AgentCore fills the sandbox seam.***

The pipeline — five components, one job each

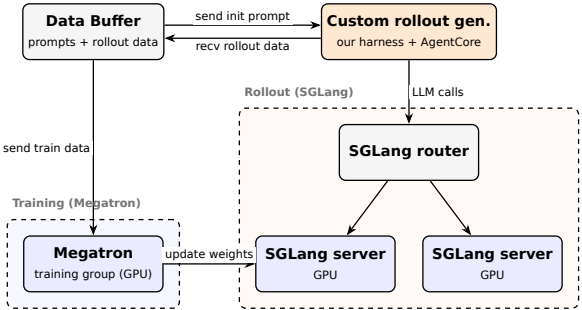
- **Trainer / Inference** (GPU) — learn, and generate turns at the *current* policy (kept fresh by **weight sync**).
- **Proxy** (CPU) — makes every model call hit the *policy under training*, not an external API.
- **Harness** (CPU) — the agent loop that owns one episode end-to-end.
- **Sandbox** (CPU/GPU) — the isolated box untrusted output runs in; the **checker scores the reward**.

Two lanes, one cycle:

- **Generation lane (→):** trainer weights → SGLang → proxy → harness turn → sandbox tool call → observation → next turn.
- **Training-data lane (←):** trajectory → reward attached → loss-masked sample → buffer → trainer step → weight sync.

AgentCore fills the sandbox seam — ad-hoc + isolated per episode. The harness needs *warmth*, not isolation, so it stays **in-process**.

slime — the trainer, and its asynchronous rollout



slime [9] = Megatron (training) + SGLang [27] (rollout), bridged by a **Data Buffer**; the agentic loop plugs into its rollout **generation hook** — no fork of the training kernel.

- **Sync mode** alternates train / rollout — each side's GPUs idle while the other runs.
- **Async driver** (`train_async.py`) overlaps them: Megatron trains batch k while the rollout side already generates batch $k{+}1$.

slime — the trainer, and its asynchronous rollout (cont.)

- **Fully-async rollout** (`fully_async_rollout`) keeps a *fixed pool of in-flight episodes across rollout boundaries*, so a step never waits on the slowest trajectory — the **long-tail** problem of agentic RL.
- **Why it matters here:** AgentCore sandboxes have **variable, sometimes long** per-episode latency; async rollout keeps the trainer fed while they run — at the cost of **staleness** (rollouts generated on an older policy version). This project runs the fully-async path (`MODE=B`).

Section 3

3. Foundations of AgentCore Runtime

AgentCore basics — three concepts, precisely

- **Runtime** — a versioned, containerized host behind an ARN (`CreateAgentRuntime` [D9]). The deployable unit; holds no per-user state.
- **Session** — one episode, addressed by a `runtimeSessionId`. First invoke cold-starts compute; the same id re-routes to that warm compute [D3].
- **microVM** — the unit of isolation: own kernel, CPU, memory, filesystem; sanitized on termination [D2].

Cardinality is 1 : N : 1 — one Runtime serves many sessions; each session = exactly one microVM. **The number of Runtimes tracks distinct sandbox *images*, not tasks.**

Concurrency — episodes, sessions & the Runtime together

`N` concurrent episodes run as `N` session ids on ONE Runtime — not `N` Runtimes.

- The `generate` hook opens **one session per episode** (`CONC` in flight at once); each session id pins that episode to **its own microVM** (or its own GPU instance).
- Every tool op of an episode carries the *same* `runtimeSessionId`, so it returns to the **same warm microVM** — state persists within the episode, isolation holds across episodes.
- The capacity provider places those `N` sessions for you; **one image (one ARN)** backs them all. Add a **second ARN** only for a different image/substrate (e.g. GPU), then round-robin the pool.

So concurrency = more sessions, not more Runtimes.

Why a microVM (not just a container)

Running **untrusted, model-generated code** — isolation is the whole point: one episode's code can't see another's files, the trainer's weights, or the host, and the box is wiped at session end.

	Docker container	**AgentCore microVM**	E2B
Boundary	shared host kernel	**own kernel** (Firecracker-class)	own kernel
Runs your image?	yes	**your OCI image in a microVM**	Dockerfile templates
Managed by	you	**AWS, in your account** (IAM/VPC)	E2B service
GPU	host-dependent	**via Instances**	not core

Two compute types — CPU microVM vs GPU Instances

CPU work belongs on the microVM; Instances exist to add a GPU [D5] (EC2 in your account via a capacity provider). **No cross-session GPU time-slicing** — scale by adding sessions/instances, not by fractioning one (no MIG on L4).

Property	Serverless microVM	Instances (GPU)
GPU	none	g4dn/g5/g6/g6e… + inf2
Max session	8 h	14 d
Isolation	1 microVM / session	1 instance / session (**1 GPU**)
Filesystem	ephemeral	persistent EBS
Agents / session	1	many

Two compute types — CPU microVM vs GPU Instances (cont.)

One session = one whole instance (session→instance is 1:1 — you get the *instance*, not a GPU). Two ids acquire two *separate* instances, so you can't split one multi-GPU box across sessions. To use every GPU of one instance: **one shared session + a self-managed in-instance Ray farm** (`demo_gpu_farm_fractional`, `demo_gpu_farm_whole`).

The microVM’s resource envelope — three limits that bite

Serverless isolation is cheap, but the microVM is **small and ephemeral**. Three limits shape every image:

Limit	Value	Why it bites	Work-around
Image size	**≤ ~2 GB** (arm64, project-measured)	a CUDA/torch wheel alone is ~2.7 GB → build fails	install heavy wheels **at `CMD`**, not baked (GPU image: ~90 s cold)
Disk	**ephemeral only** — no persistent volume [D3]	a `git clone`d repo / installed pkg is **wiped at session end**	durable state → S3 / session Memory, or move to **Instances** (EBS) [D5]
vCPU / RAM	**metered, not sized** (vCPU-hr + GB-hr) [D13] — no fixed knob	a data-heavy checker can exceed the account quota	check **Service Quotas**; big fixed footprint → **Instances** (EC2 spec)

The microVM’s resource envelope — three limits that bite (cont.)

- **Example:** the g6 sandbox image can’t bake torch (>2 GB) → `pip install torch` at `CMD` so the wheel lands in the ephemeral workspace at boot; a big SWE repo + persistent workspace → move to **Instances** (EBS, 14 d).
- **Rule of thumb:** *files-in-a-session* for light deps → *CMD-time install* for medium → *Instances* when the footprint is large and **fixed**.

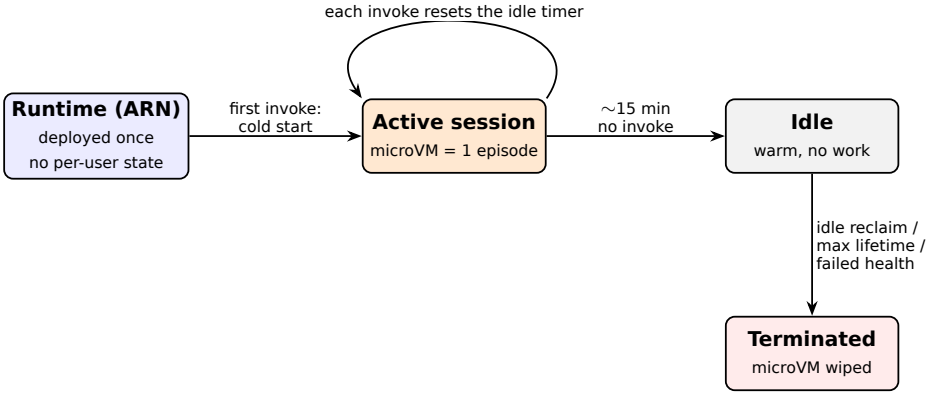
Building & deploying a Runtime — three ways, one contract

Contract [D17]: ARM64 image, serve `0.0.0.0:8080` with `/invocations` + `/ping`. Three ways to put your code on a Runtime (increasing control): the **`@aws/agentcore` CLI** [D16] (zero-Docker zip) → the **`bedrock-agentcore` SDK** [D15] (`@app.entrypoint` implements the contract) → **bring-your-own container** (max control — this project).

```
FROM public.ecr.aws/docker/library/python:3.12-slim   # CPU sandbox
RUN pip install --no-cache-dir bedrock-agentcore
COPY agentcore_slime_demo/sandbox_server_agent.py /app/
CMD ["python","-m","agentcore_slime_demo.sandbox_server_agent"]  # serves :8080
```

- **GPU image:** point `LD_LIBRARY_PATH=/usr/lib64` at the injected `libcuda.so.1`, and `pip install torch` **at `CMD`** (not baked) to stay under the **2048 MB** cap.
- **Deploy:** ECR create → `buildx --platform` (arm64 microVM / amd64 GPU) → push → `CreateAgentRuntime` → (GPU) `CreateCapacityProvider` (`g6.xlarge`, VPC, `idleInstanceTimeout=600`) + attach.

Session lifecycle — launch, idle timer, terminate



*Launch is **implicit** (first invoke); terminate is **implicit** (idle / max-lifetime / health). A heartbeat ping resets the timer (HEARTBEAT=1, any demo).*

How to launch & terminate a session — the mechanics

- **Launch = the first `invoke_agent_runtime` on a fresh `runtimeSessionId`.** There is no explicit "start" call; the id (≥33 chars) *is* the launch. The first op cold-starts the microVM (or GPU instance — minutes on a cold GPU); every later op with that id re-routes to the same warm box.
- **Terminate = passive.** No explicit stop on the happy path — the session ends on **idle reclaim (~15 min)**, **max lifetime** (8 h microVM / 14 d instance), or **failed health**. Each invoke resets the idle timer.
- **Consequence in the RL loop:** a **100-row dataset does not launch 100 instances** — only `CONC` episodes are in flight, recycling through rows over steps. **Concurrency multiplies instances, not data.**
- The **weight-sync pause can trip the idle clock** and silently lose a warm session → the resilience levers in §5.
- **Latency: measure, don't assume** — every op is instrumented (`sandbox_ms`, `sglang_gen_ms`).

Why AgentCore vs. rolling your own sandbox

Every agentic-RL team must solve “run untrusted code, isolated, at scale.” The options and the trade:

	Self-managed K8s / Firecracker	E2B / Modal (SaaS)	**AgentCore Runtime**
Isolation	microVM (you operate)	microVM (their cloud)	**microVM, per session**
In your account / VPC	yes — you build it	**no** — their cloud	**yes** — IAM/SigV4, VPC
Bring-your-own container	yes	limited	**yes** (OCI image)
GPU sandbox	you wire it	limited / none	**yes, on Instances**
Ops burden	**high** (run the fleet)	low	**low** (managed)
Session length	you choose	provider cap	8 h µVM / **14 d Instance**

Why AgentCore vs. rolling your own sandbox (cont.)

The client point: AgentCore gives SaaS-level "no fleet to run" ***without*** *your code or data leaving your account.*

Security & isolation — safe to run untrusted output

Every episode executes model-generated code you did not write. AgentCore makes that safe **by construction**:

- **Per-session microVM isolation** — a hardware-virtualized boundary (not a shared container); one episode's code cannot see another's.
- **In your account, in your VPC** — the sandbox runs under **your IAM**, authenticated with **SigV4**; VPC-mode Runtimes keep traffic on **private endpoints** — **nothing leaves your account**.
- **Bring-your-own image** — you own every dependency, user, and filesystem, and control what the sandbox can reach on the network.
- **Ephemeral by default** — a fresh session per episode, reclaimed on idle; no state bleeds across rollouts unless you deliberately snapshot it (`gpu_single` `SNAPSHOT_EVERY=1`).

This is the enterprise reason to run agentic RL on AWS rather than a third-party sandbox cloud.

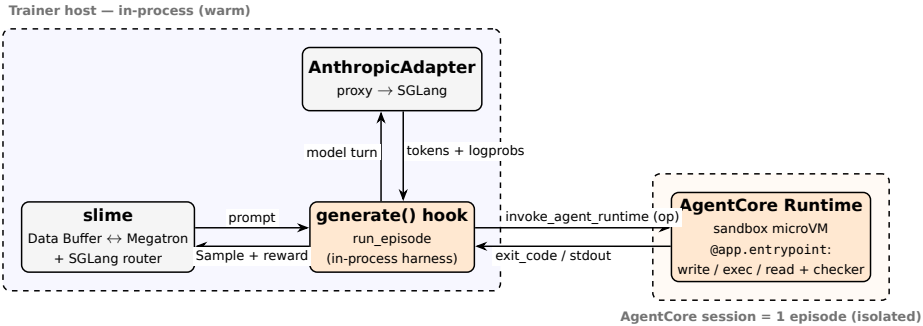
Cost & economics — what you pay, and the packing win

- **Billing model:** you pay for sandbox **vCPU-seconds + GB-hours** while a session is live [D13], plus **Instance** hours for GPU sandboxes — not a flat per-sandbox fee. Idle sessions are reclaimed (~15 min), so **warm-but-unused time is bounded**.
- **CPU sandboxes are cheap; the bottleneck is elsewhere:** for cheap-checker families (math, code, SQL, tool-use) the measured sandbox op is **~0.1 s** (p50 123 ms / p95 900 ms) — generation dominates, so you scale inference, not sandboxes.
- **GPU is where cost concentrates — and where packing pays:** KernelBench is sandbox-bound (exec p50 **8.5 s** ≫ generation 2.5 s). Isolated needs **N whole g6 at once**; the Ray farm (`demo_gpu_farm_fractional`) packs **N episodes onto one GPU** (`num_gpus=0.5` → 2×), so you provision **one** instance, not N.
- **Primary lever is capacity, not just cost:** on a capacity-starved GPU fleet you often *cannot* launch N instances at all — the farm makes the run *possible*, and cheaper.

Section 4

4. Integrating AgentCore into slime as the sandbox

How AgentCore sessions integrate into slime — the call path



*One `generate()` hook, one **`runtimeSessionId` per episode**; each tool call = one SigV4 `invoke_agent_runtime`.*

Plugging into slime — one hook, no fork

slime exposes the rollout as a **custom generation function**, so the agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path ⋯generate.generate` — drives the in-process harness; each tool call → one `invoke_agent_runtime` keyed by that episode's **`runtimeSessionId`**.
- **Design choice:** AgentCore is the **sandbox, not the harness**. The harness stays in-process (warm); the sandbox is the isolated per-session microVM. (It *can* host the harness too — `demo_cpu_branching` runs the real Claude Code CLI inside a Runtime.)
- The returned **loss-masked `Sample` + reward** flows straight back into slime's buffer — no fork of Megatron/SGLang.

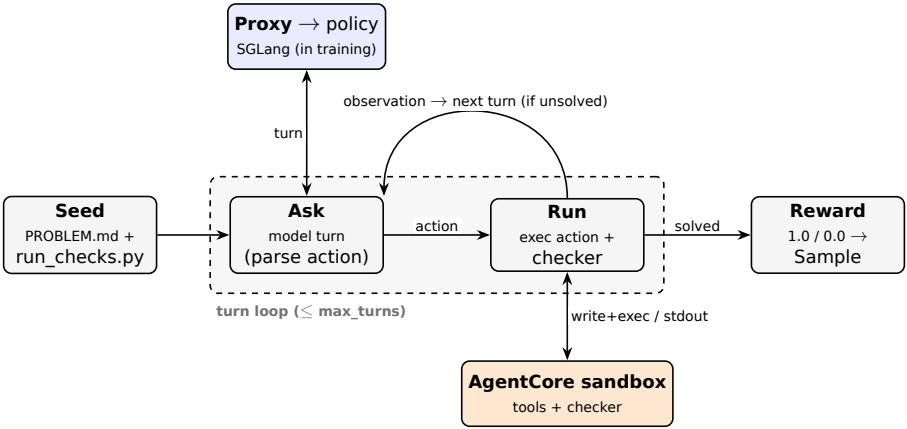
Who owns the lifecycle

slime's training kernel controls **neither** the Runtime nor the session.

- **Runtime** (create/delete the ARN) — the **deploy driver**, once before training. slime just gets the ARN.
- **Session** (per episode) — the `generate` hook's sandbox client: **launch implicit** (first invoke cold-starts the microVM); **terminate implicit** (idle reclaim, no explicit stop on the happy path).

Because termination is *passive*, the weight-sync pause can trip the idle clock and lose a warm session — the failure mode the `HEARTBEAT=1` knob (any demo on an own Runtime) addresses.

The episode, end to end — the loop



Seed → Ask (proxy) → Run (sandbox + checker) → Reward; unsolved loops back.

Step-by-step — one MBPP episode

1. **Deploy (once):** build the arm64 sandbox image → `CreateAgentRuntime` → get the ARN (the deploy driver, not slime).
2. **Launch run:** `run.sh` sets `SANDBOX=agentcore`, `FAMILY`, `CONC`, `EPISODE_MAX_TURNS`, then `ray job submit ⋯ train_async.py --custom-generate-function-path ⋯generate.generate`.
3. **Open session:** slime hands a prompt to `generate()`, which rebuilds the `Task` and calls `make_sandbox("agentcore")` → **cold-starts** that episode's microVM (`runtimeSessionId`).
4. **Seed:** `write_file` `PROBLEM.md` + hidden `run_checks.py` into the session.
5. **Turn loop (≤ `EPISODE_MAX_TURNS`):** `policy.complete` via the **proxy** (TITO tokens) → extract `solution.py` → `exec("python3 run_checks.py")` in the sandbox → on failure, feed stdout back.
6. **Reward & learn:** `exit_code == 0` → **reward 1.0/0.0** from the in-sandbox checker → `build_token_stream` → loss-masked `Sample` → buffer → Megatron step → **weight sync** to SGLang.

Key code (1/2) — the in-process generate hook

slime calls this once per prompt: build the policy at the SGLang router, open **one sandbox per episode**, run the loop, return one loss-masked `Sample`.

```python
# agentcore_slime_demo/generate.py   (--custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    policy  = SGLangPolicy(router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_po
                           model_path=args.hf_checkpoint)
    task    = _task_from_sample(sample)            # rebuild Task from row metadata
    sandbox = make_sandbox(os.environ.get("SANDBOX", "agentcore"))
    async with sandbox as sb:                      # one session == one episode
        traj = await run_episode(policy, sb, task, sampling_params)
    tokens, resp_len, loss_mask, logprobs = build_token_stream(traj, tokenizer)
    return Sample(tokens=tokens, response_length=resp_len, reward=traj.reward,
                  loss_mask=loss_mask, rollout_log_probs=logprobs)
```

- Loss mask = **1 on the policy's own tokens, 0 on prompt / observation / tool tokens**.
- `build_token_stream` reuses the **recorded `token_ids` per turn** — no re-tokenization (TITO); returns a *bare* `Sample` (slime's async path chokes on nested lists).

Key code (2/2) — the sandbox client & the Runtime server

Each `write_file` / `exec` / `read_file` = **one SigV4 `invoke_agent_runtime`** [D8] keyed by the episode's session id; boto3 offloaded with `asyncio.to_thread` so N episodes run concurrently (with retries for transient blips + the GPU cold launch).

```python
# client — agentcore_runtime_sandbox.py
def _invoke_sync(self, payload):
    resp = self._client.invoke_agent_runtime(agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id,        # >=33 chars; one per episode
        payload=json.dumps(payload).encode())
    return json.loads(resp["response"].read().decode())

# server (deployed image) — the checker runs HERE, so the microVM is the reward source
app = BedrockAgentCoreApp()
@app.entrypoint
def entrypoint(request):        # dispatch on op: ping / write_file / read_file / exec
    return handle(request)
app.run()                       # serves 0.0.0.0:8080  (/invocations + /ping)
```

CPU → GPU is one env var (`SANDBOX=`) — zero change to the loop.

One image, many tasks & dependencies

One microVM = one image per session. Task diversity is absorbed *inside* one generic image, in increasing order of how structural the difference is:

1. **Files, not images** — most CPU families (math `sympy`, code `unittest`, SQL `sqlite3`, tool-use, light CTF) share **one generic Python image**; prompt, seed files, and hidden checker are written into *that episode's session*.
2. **Per-session install** — heavier deps go into the ephemeral workspace: **`CMD`-time** (the GPU image installs the ~2.7 GB torch wheel at boot to stay under the ~2 GB arm64 cap) or **seed-time** `git clone`/`pip`.
3. **Per-capability image** — only a genuine substrate difference (chiefly the **GPU** toolchain: `nvcc` + a real GPU, or a prebuilt dependency set) needs its **own Runtime ARN**, routed by `task.image` (`demo_cpu_terminalbench` at one image per task, the GPU demos). Most CPU families need none: the default Code Interpreter already has pandas / scipy / sklearn.

Section 5

5. The demos — one storyline

Find your workload — the demos are client use cases

Each demo stands in for a **real agent a client would train**. Find yours:

- **Coding / math / SQL / RAG / tool use / data science** → `demo_cpu_tasks` (`DATASET=mbpp`, `humaneval`, `gsm8k`, `math500`, `deepscaler`, `gretelsql`, `hotpotqa`, `xlam`, `mcp`, `libs`, `mixed`) — on the **default Code Interpreter, no image**.
- **Terminal / DevOps / SWE agents that act on the whole OS** → `demo_cpu_terminalbench` (one image per task).
- **A checker with a > 2 GB toolchain** → `demo_cpu_instance_pool` (warm m5 Instances, isolated sessions).
- **A high-throughput rollout fleet** → `demo_cpu_scale_lanes` (4 engines : 256 harness workers : 256 microVMs, decoupled).
- **GPU kernels / systems** → `demo_gpu_single`, `demo_gpu_farm_fractional`, `demo_gpu_farm_whole`.
- **A real coding-agent CLI as the harness** → `demo_cpu_branching` (experimental).

The storyline — 7 demos + 1 experimental, 8 chapters

1. **The tasks** — dataset, URL, env deps, image or not.
2. **Most tasks train on the default Code Interpreter**, no image (`demo_cpu_tasks`).
3. **Custom images at scale** — one image + Runtime per task (`demo_cpu_terminalbench`).
4. **Hiding the container launch** — pre-open + heartbeat (`warmstart/` probe).
5. **A warm CPU Instance pool, shared with isolation** (`demo_cpu_instance_pool`).
6. **Scaling the rollout on decoupled tiers** (`demo_cpu_scale_lanes`).
7. **GPU sandboxes + GPU isolation** (`demo_gpu_single`, `demo_gpu_farm_fractional`, `demo_gpu_farm_whole`).
8. **Experimental** — one episode, many samples, real Claude Code CLI (`demo_cpu_branching`).

*Policy **Qwen3-4B**, trainer **slime**, harness **in-process**; the same six step scripts everywhere. Every run is a **smoke** (10 train steps ·1 eval) — not convergence.*

All demos at a glance (1/2) — CPU

Knobs, not demos: `HEARTBEAT=1` (every demo), `compare_backends.py` (`demo_cpu_tasks`), `SNAPSHOT_EVERY=1` (`demo_gpu_single`), and the decided `SLIME_CUDA_VISIBLE` / `SLIME_NUM_GPUS` pin (unset = 8-GPU placement).

Ch.	Demo	Sandbox substrate	Task	Status
1–2	`demo_cpu_tasks`	managed **Code Interpreter** (default)	any literature RLVR dataset, or a mix	**passed** on CI (2026-09-25): gsm8k 0.90, libs 1.0
3–4	`demo_cpu_terminalbench`	µVM, one image + Runtime **per task** (218)	Terminal-Bench	**passed** (2026-09-23): boot hidden, 4/382 solved
5	`demo_cpu_instance_pool`	warm **m5 Instance** pool, uid per episode	DeepScaleR, > 2 GB image	pool + isolation probes passed; RL smoke **passed** 2026-09-25, eval 0.7875, 2 cold boots
6	`demo_cpu_scale_lanes`	256 pooled µVM sessions	DeepScaleR (fixed)	pool check ok; [PENDING: RL smoke] (old 4×4×4 lanes: eval 0.358)

All demos at a glance (2/2) — GPU and experimental

Ch.	Demo	Sandbox substrate	Task	Status
7	`demo_gpu_single`	one `g6.xlarge` (L4) per session	KernelBench	**trained**; [PENDING: root cause of the op=open 500s]
7	`demo_gpu_farm_fractional`	shared GPU instance, 0.5 + 0.5 of **one** GPU	KernelBench	**passed** (2026-09-25): eval 0.65
7	`demo_gpu_farm_whole`	shared GPU instance, one **whole** GPU per episode	KernelBench	passed (2026-09-21): 0.65 / 0.70
8	`demo_cpu_branching` *(exp.)*	µVM with node + `claude`	SWE bug-fix, real Claude Code CLI	unblocked; [PENDING: branching RL smoke fan_out and reward]

Ch. 1 — the tasks (1/3): code and math

Every task is a real RLVR dataset, scored by a programmatic checker. The question: does it need a custom image?

DATASET	Dataset (URL)	Train / eval	Env deps	Image?
`mbpp`	google-research-datasets/mbpp	500 / 8	stdlib	no
`humaneval`	openai/openai_humaneval	140 / 24	stdlib	no
`gsm8k`	openai/gsm8k	200 / 40	stdlib	no
`math500`	HuggingFaceH4/MATH-500	160 / 40	stdlib	no
`deepscaler`	agentica-org/DeepScaleR-Preview-Dataset	400 / 40	stdlib	no

Ch. 1 — the tasks (2/3): SQL, RAG, tools, data science

DATASET	Dataset (URL)	Train / eval	Env deps	Image?
`gretelsql`	gretelai/synthetic_text_to_sql	300 / 40	`sqlite3`	no
`hotpotqa`	hotpotqa/hotpot_qa (distractor)	300 / 40	10 passage files	no
`xlam`	minpeter/xlam-function-calling-60k-parsed	300 / 40	stdlib	no
`mcp`	Team-ACE/ToolACE (MCP-style)	300 / 40	stdlib	no
`libs`	synthetic datasci + scipy + sklearn	72 / 6	pandas, scipy, sklearn	**no** — preinstalled
`mixed`	synthetic math + SQL + CTF + datasci	174 / 9	stdlib + pandas	no

Ch. 1 — the tasks (3/3): where a custom image IS needed

The pattern: literature datasets need only Python + its scientific stack → the **default sandbox**. A custom image only for a **pinned OS**, **> 2 GB**, **CUDA**, or a missing toolchain (CI: **no git, no internet**).

Task (chapter)	Dataset (URL)	Train / eval	Env deps	Image?
Terminal-Bench (Ch. 3)	laude-institute/terminal-bench	135 / 31 usable	a **pinned OS per task**	**one per task**
DeepScaleR, heavy stack (Ch. 5)	DeepScaleR (above)	400 / 40	torch + sci stack, **> 2 GB**	**CPU Instance**
KernelBench (Ch. 7)	ScalingIntelligence/KernelBench	100 L1 / 20 L2	CUDA + `nvcc` + torch	**GPU Instance**
SWE bug-fix (Ch. 8)	synthetic SWE tasks	64 / —	node 22 + `claude` CLI	yes

Ch. 2 — demo_cpu_tasks: most tasks train on the default runtime

Use case covered. RLVR on the literature's datasets — code, math, SQL, RAG, tool calling, data science — alone or mixed.

Why this demo. It answers *do I have to build a sandbox image?* Mostly **no**: the Code Interpreter has the deps — no build, deploy or cleanup. One demo, not one per dataset: same harness, sandbox, trainer.

Design considerations.

- **CI by default**, own Runtime opt-in; steps 2/3/4/6 are no-ops.
- **One harness**: self-describing rows rebuild each task; mixes are GRPO-safe (per-group advantage).
- **Heartbeat is a no-op on CI**; the lever is `AGENTCORE_CI_SESSION_TIMEOUT`.
- **Rejected:** a per-library image pool — CI already has pandas / scipy / sklearn.

Ch. 2 — how it runs, and the CI quotas

- **Per episode:** start session → `write_file` / exec / `read_file` → stop; its own microVM, no ARN, no IAM role.
- **Env (probe 2026-09-25):** Python 3.12, numpy, pandas, scipy, sklearn, sympy, torch-CPU, sqlite3, gcc, node — **no git, no internet**. All **13 families** score 1.0.

The invoke rate binds first: ~8 ops per 20 s episode → ~**75 episodes in flight**; raise it before `CONC` ≥ 32.

Quota (us-west-2 default)	Code	Default	This demo
Concurrent CI sessions	L-CAF6F552	1000	`CONC` × engines = **8**
`StartCodeInterpreterSession`	L-6B09F374	30/s	~1 per episode
`InvokeCodeInterpreter`	L-04A719E9	30/s	~6–10 per episode

Ch. 2 — measured on the Code Interpreter (2026-09-25)

- **`gsm8k` — PASSED: `eval/gsm8k` 0.90**; 425 CI sessions, **0 infra errors**; step ~42–54 s.
- **`libs` — PASSED, no image: `eval/libs` 1.0**, `raw_reward` 1.0 on 9/10 steps; datasci 132 / scipy 109 / sklearn 108 episodes; **1883 ops, all ok**.

Caveat: both near-saturated → a **plumbing proof, not a learning curve**. Session open (~1.8 s) is the largest sandbox cost.

op (libs run)	count	p50	p95
`start_session`	350	1.83 s	3.50 s
`write_file`	834	69 ms	127 ms
exec (imports pandas/scipy/sklearn)	353	1.34 s	2.10 s
`stop_session`	346	150 ms	1.34 s

Ch. 2 — earlier dataset smokes on the own-Runtime path (2026-09-18→19)

Eleven 10-step smokes, held-out solve rate — checker and reward are identical on both backends (`compare_backends.py` asserts it):

- **Code:** `mbpp` 0.69 ·`humaneval` 0.85
- **Math:** `math` 1.00 ·`gsm8k` 0.90 ·`math500` 0.89 ·`deepscaler` 0.78
- **SQL / RAG:** `gretelsql` 0.60 ·`hotpotqa` 0.65
- **Tool use:** `xlam` 1.00 ·`mcp` 1.00
- **Mix:** `mixed` 0.67

Re-running them on the Code Interpreter: [PENDING: CI smokes for the other DATASET values].

*Cheap checkers (~0.1 s sandbox op vs ~1.8 s decode) make all of these **generation-bound**: scale inference, not sandboxes.*

Ch. 3 — demo_cpu_terminalbench: custom images at scale

Use case covered. Agentic terminal and SWE-style tasks where every task pins its own OS image — Terminal-Bench's ~229 containerized tasks [6], graded by each task's hidden tests.

Why this demo. The other side of Ch. 2: when a task *does* need a custom image, how do you run **hundreds**? One image + one Runtime per task, strict per-task routing. Its own demo: per-task images, a fresh microVM per episode and a tool-calling loop all differ from `demo_cpu_tasks`.

Design considerations.

- **A fresh microVM per episode, never reused** — the tasks mutate the OS.
- **Oracle validation before training** — drops the 47 tasks whose reference fails here.
- **Look-ahead pre-open** hides the cold start (Ch. 4); **heartbeat always on**.
- **Single-container tasks only** (one image ↔ one sandbox).
- **Quota:** pre-opened + running sessions vs Active Session Workloads (5000).

Ch. 3 — the pool and the smoke

- **The pool:** one arm64 image (ECR `acs-tb-*`) + one Runtime (`acs_tb_*`) per task → `tasks/arn_by_image.json`. **218 / 229** deployed; oracle check (untouched env fails, reference passes) → **166 usable** = 135 train / 31 eval.
- **Strict routing:** `task.image` → `AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE`; an unmapped task is an **error**, never a fallback Runtime.
- **The loop:** tool-calling `MiniClaudeHarness` (SGLang qwen25 tool parser); reward = the task's hidden oracle, run by `grade.sh` from `/opt/acs/oracle`, outside the workdir.
- **Smoke — PASSED (2026-09-23):** 382 episodes in ~70 min, 0 infra errors.
  - **Cold start fully hidden:** `open_wait_ms` p95 **0**, 100 % warm hits.
  - **The price:** pre-opened sessions idled p50 7.3 min vs a 21 s episode → **~95 % idle**.
  - **Reward sparse** for a 4B policy: 4 solves / 382; 42 % of episodes pass ≥ 1 test.
- **Next:** `TB_PREOPEN_BATCHES=1` (cost), an easy-first curriculum or partial-credit reward (signal).

Ch. 4 — pre-open and heartbeat: the `warmstart/` probe

Use case covered. Any rollout that opens a fresh sandbox per episode (a container launch each time), and any run whose sandboxes sit idle across the weight-sync pause.

Why this probe. The Terminal-Bench smoke showed pre-open works but wastes ~95 % of booted time — too costly to tune at full scale (~70 min, 229 Runtimes). The probe answers *how much does pre-open save?* and *does a heartbeat keep a session alive?* in ~7 min for a few cents. A probe folder inside `demo_cpu_terminalbench`, not a demo: it tests that demo's two mechanisms.

Design considerations.

- **MockPolicy episodes**, no GPU / model / trainer → only the sandbox is measured.
- **A 60 s idle timeout** makes reclaim observable in minutes.
- **Both backends** (own Runtime and Code Interpreter): they behave differently under a heartbeat.
- **Pre-open depth** 1 batch here; `TB_PREOPEN_BATCHES` (default 2) in the real demo.

Ch. 4 — how the look-ahead pre-open works

- **The problem:** slime hands the pool only `CONC × engines` episodes and draws the next batch only when this rollout ends — the pool alone cannot see ahead.
- **The fix:** slime's prompt order is **deterministic** (a shuffled dataset walked in order, fixed seed per epoch). A **look-ahead data source** (`tb_lookahead_source.py`, `--data-source-path`) replays it and, on every draw, tells the pool the episodes of the **next `TB_PREOPEN_BATCHES` batches**.
- **The pool** keeps `TB_PREOPEN_BATCHES × RB × NS` = 2 × 4 × 8 = **64** upcoming episodes booted, each on its own new session on its task's Runtime; a worker claims a ready one and the next starts booting.
- **Mispredictions** are trimmed at the next draw and open on demand. `TB_PREOPEN_BATCHES=0` turns it off.
- **Heartbeat:** a `SessionKeeper` pings (`exec("true")`) any session idle past `WARM_PING_FRACTION × WARM_IDLE_TIMEOUT` (0.5 × 900 s). Always on here; `HEARTBEAT=1` (default off) in every other demo.

Ch. 4 — measured: pre-open

2026-09-25, us-west-2: 24 MockPolicy episodes in batches of 6.

- **Pre-open takes the boot off the critical path**; only the first batch pays it.
- **The cost is idle held** — Terminal-Bench's 95 % idle at scale → TB_PREOPEN_BATCHES=1 next.

Backend	Mode	Wait p50	Waited	Idle held	Makespan
own Runtime	cold	774 ms	24/24	0 s	31.9 s
own Runtime	pre-open	**5 ms**	8/24	75 s	25.5 s (**−20 %**)
Code Interpreter	cold	1715 ms	24/24	0 s	28.7 s
Code Interpreter	pre-open	**0 ms**	6/24	70 s	24.3 s (**−15 %**)

Ch. 4 — measured: heartbeat

3 sessions idle 150 s against a 60 s idle timeout.

- **Only helps an own Runtime:** a reclaimed session *looks healthy* but lost its files — silent; so Terminal-Bench's keeper is always on.
- **No-op on the Code Interpreter:** `run.sh` turns it off; the lever is `AGENTCORE_CI_SESSION_TIMEOUT` (3600 s default, free: sessions stop with their episode).

Backend	Heartbeat	Survived	Resume
own Runtime	on (14 pings)	**3/3**	104 ms, files intact
own Runtime	off	0/3	**silently reclaimed**: same id, fresh empty sandbox
Code Interpreter	on or off	0/3	expires at its session timeout regardless

Ch. 5 — demo_cpu_instance_pool: warm m5 pool

Use case covered. Checkers needing a big image (> 2 GB, torch + sci stack here), which must run on a CPU **capacity-provider Instance** (`m5.xlarge`), not a microVM.

Why this demo. A ~80 s cold boot is too costly per episode, so sessions share a few warm Instances — *so how is isolation kept?* Its own demo: substrate, sharing model and isolation story all differ from the microVM demos.

Design considerations.

- **`SANDBOX_POOL=N`** (default 2), least-loaded, one Runtime; idle Instances reclaimed at 600 s.
- **Per-episode `sub_session` as its own uid**, kill-and-scrub on close.
- **Honest limits:** one kernel, netns, CPU/RAM — hostile code needs a microVM.
- **Rejected:** an Instance per episode (~32 boots/step); one warm Instance (no parallelism).

Ch. 5 — the warm pool, and what it measured

- **How:** `SANDBOX=agentcore-ray-farm SANDBOX_POOL=N` mints N pool session ids per run (`cpupool-<run>-m00 ⋯`). AgentCore pins each `runtimeSessionId` to its own Instance → **N warm m5s**, cold boot paid **N times per run**, not once per episode.
- **Per episode:** assigned to the least-loaded pool member, runs as its own `sub_session`; closing it never closes the pool session.
- **Why it matters:** the old per-episode-Instance run paid ~**32 cold boots per step** (375 total; `eval/deepscaler` 0.76); a single warm Instance cut 201 boots to 1.
- **Measured:**
  - offline: 9 episodes split 5 / 4 over two members, load back to 0;
  - **live probe: 2 distinct Instances**, **2 cold boots** (83.1 s, 83.6 s, in parallel), then warm opens **121–328 ms**, reward 1.0 × 6;
  - isolation probe **16/16 offline**; image `acs-sandbox-bigcpu:uidiso-20260925`;
  - RL smoke: `smoke_pool2_20260925_113657`: 10 steps, job succeeded; 406 episodes on exactly 2 Instances (210 / 196), **2 cold boots** (only 4 episodes in step 1 waited, 63 s); warm op p50 117 ms / p95 261 ms; step time 38–219 s vs ~715–753 s before pooling; `raw_reward` 0.72–1.0; **`eval/deepscaler` 0.7875**.

Ch. 5 — the warm pool, and what it measured (cont.)

Ch. 5 — m5 isolation (1/3): files, processes, RAM

A microVM gives one kernel per session. A shared m5 gives that up for warm reuse, so the farm puts back what one kernel allows: every `sub_session` runs as its own unprivileged uid.

Layer	microVM per session	shared m5 — **enforced**	shared m5 — **still shared**
Filesystem	own root fs	own `0700` dir owned by the episode uid; root `0711`; `umask 077`; HOME/TMPDIR inside; no `..` / absolute / symlink escapes	read-only system dirs
Process / user	own tree	uid `20000+slot`, no extra groups: no ptrace, signals or `environ` of another episode	one PID namespace (ps)
RAM / CPU	own allocation	`prlimit` NPROC 256, FSIZE 1 GiB, CORE 0; per-op timeout kills the tree	no cgroup: an OOM can hit a neighbour

Ch. 5 — m5 isolation (2/3): network, kernel

- **Clean-up on close:** SIGKILL every process of the uid (daemons too), reap zombies (subreaper), remove the dir and the uid's files in `/tmp`, `/var/tmp`, `/dev/shm`; open scrubs a reused slot.
- **The `sub_session` id is the key:** kept off every command line, regex-validated against traversal.

Layer	microVM per session	shared m5 — **enforced**	shared m5 — **still shared**
Network	own interface	the server's `AWS_*` env is stripped	one netns: port 8080 and IMDS reachable; another episode needs its 64-bit id
Kernel	own guest kernel	—	**one kernel**: an exploit breaks everything above
Blast radius	one session	—	**one instance**

Ch. 5 — m5 isolation (3/3): verified; when enough

- **Verified — `isolation_probe.py`:** episode A plants a secret file, a `/tmp` file and a detached process; episode B tries to reach them.
  - **Before hardening:** B read A's file by path (`cat ../<A>/marker.txt`) — dir hygiene, not a boundary.
  - **After: 16/16 checks pass offline** (container and built image): no listing, no read by any path, no signal, no `AWS_*`; after close A's process and `/tmp` file are gone; a fork loop stops at EAGAIN, a 2 GB write at 1 GiB.
  - Live on the updated Runtime: [PENDING: live isolation_probe.py –live result on the updated Runtime].
- **When is it enough? Good enough for RLVR**, including reward hacks: reading a sibling's `solution.py`, killing a neighbour's checker, a daemon that tampers with the next episode.
- **Not** a defence against a kernel exploit, and no guaranteed CPU/RAM or private network. For those — and for OS-mutating tasks (Terminal-Bench) — use a **microVM per session**.

Ch. 6 — demo_cpu_scale_lanes: decoupled rollout tiers

Use case covered. High-throughput rollout: a fleet that has scaled inference and the sandbox and now hits the forgotten third tier, the harness driver.

Why this demo. It shows `throughput = min(inference, sandbox, harness)` with **4 engines : 256 harness workers : 256 microVMs** as independent pools. Harness and sandbox scale for different reasons (Python concurrency vs quota and cost), so they are two knobs, not one. Its own demo: it holds task and image fixed and varies only the topology.

Design considerations.

- **Joined per episode:** a free worker pulls it → borrows **any free microVM** → takes the **least-loaded engine** (sticky for its turns). Nothing pinned.
- **Isolation:** one episode per session at a time; fresh ep-<hex> dir, `rm -rf` before reuse; `SANDBOX_SESSIONS=0` for OS-level tasks.
- **256 opens** at 20/s (quota 25/s); client pools sized to 256; `RB × NS = 256` episodes/step.

Ch. 6 — measured

- **Live pool check (2026-09-25, no trainer, W = M = 64, real Runtimes):** 1280 ops ok; engines **48 / 48 / 47 / 49**; a worker used up to 2 sessions and a session served up to 2 workers, so the tiers mix. [PENDING: 4 : 256 : 256 RL smoke]
- **Old 4 × 4 × 4 lanes on AWS (2026-09-23):** 4 opens for 400 episodes; `raw_reward` 0.19–0.50, **`eval/deepscaler` 0.358**.
- **Offline sweep (`harness_scaling_demo.py`) — the `min()` law:** throughput 5.3 / 10.5 / 20.9 / 29.6 / 36.5 ep/s at `W` = 1 / 2 / 4 / 6 / 8 while **harness-bound**, then a **plateau ~37 ep/s** once **inference-bound**. Unseeded sim — shape, not absolute rate.
- **Verifiable in logs:** every `[demo]` line carries `worker=`, `engine=`, `sandbox_session=`; the binding tier shows as `qdepth` (harness) / `sandbox_wait_ms` (sandbox) / `engine_inflight` (inference).

Ch. 7 — GPU: how many episodes share a GPU?

- **GPUs exist on AgentCore only on capacity-provider Instances, and one session = one whole Instance.** The managed Code Interpreter has no GPU (`cuda_available=False`; `True` on a custom `g6` Runtime).
- **All three train KernelBench:** the policy writes a CUDA `ModelNew`; the checker asserts `allclose` vs the reference and reports a speedup. Farms serve from the box's own GPUs over the same `/invocations` contract.

Demo	Episodes per GPU	Boundary between episodes
`demo_gpu_single`	one session = one g6	**hardware** (separate instance)
`demo_gpu_farm_fractional`	two per GPU (0.5 + 0.5)	software, shared card
`demo_gpu_farm_whole`	one whole GPU each, one shared instance	software, own card

Ch. 7 — demo_gpu_single: one GPU per session

Use case covered. GPU-in-the-loop verification: RL on CUDA kernels (KernelBench [7]), where the reward requires running untrusted code on a real GPU, isolated from the training state.

Why this demo. A GPU exists only on Instances. This is the simplest GPU sandbox — one custom Runtime, one `g6.xlarge` (L4), nothing shared on the GPU — and the baseline the two farm demos pack against.

Design considerations.

- **Capacity provider `acs_gpu_cp`**, warm-shared session: the ~207 s cold boot is paid once, reopens ~150–190 ms.
- **torch pip-installed at container start** (~90 s) to stay under the image cap.
- **The trainer box keeps its GPUs for slime** (4 train + 4 inference); only the checker is remote.
- **`SNAPSHOT_EVERY=1`**: application-level workspace snapshot to a store you own — no usable VM/EBS GPU snapshot.

Ch. 7 — demo_gpu_single: measured

- **Committed 10-step run:** raw_reward ~0.28–0.94, **eval/kernelbench 0.65**; exec p50 8.5 s ≫ generation 2.5 s → **sandbox-bound**.
- **Earlier smokes (knobs of this demo):** GPU-farm 0.525; SNAPSHOT_EVERY=1 0.625 (3 files saved and restored per episode); custom-Runtime angle 0.65.
- **Remote smoke (2026-09-25)** completed **4 steps:**
  - raw_reward 0.531, 0.469, 0.25, 0.5625;
  - one 207 s cold boot, then warm opens 150–190 ms; sandbox op p50 148–177 ms.
- It then **failed** (rc=1) on InvokeAgentRuntime 500s during op=open retries: [PENDING: root cause of the demo_gpu_single remote-smoke op=open 500s].

Ch. 7 — demo_gpu_farm_fractional: 2 per GPU

Use case covered. Many light GPU episodes — small kernels that use a fraction of a card — packed onto one GPU.

Why this demo. Utilization, cost, capacity: a g6 per episode needs N instances at once (stalls on a starved fleet); this needs one GPU. Separate from `_whole`: opposite trade-offs (shared card and fault domain).

Design considerations.

- **The farm:** one Runtime, one shared session id; Ray leases a 0.5-GPU actor per `sub_session`.
- **Memory cap is cooperative** (`B4_PER_SANDBOX_MEM_FRACTION=0.5`).
- **No MIG on L4 / L40S** → one shared fault domain; `nvcc flock` + `fraction=0.5` mitigate the compile wedge.
- **Admission control:** a client-side gate caps opens at the farm's slot count.

Ch. 7 — demo_gpu_farm_fractional: measured

- **Job succeeded, eval/kernelbench 0.65** (0.775 on 2026-09-21); ~83 min, slime on GPUs 2–5.
- raw_reward 0.72, 0.59, 0.22, 0.69, 0.72, 0.19, 0.72, 0.75, 0.63, 0.44.
- **Packing works:** 359 episodes split **181 / 178** across the two actors on one GPU; lease wait always 0.
- **Latency:** exec p50 9.0 s / p95 19.8 s; warm open 4.2 ms.
- **Checker outcomes:** PASSED 325 (mean 0.64), IMPORT_ERROR 18, NO_OUTPUT 16.

Ch. 7 — demo_gpu_farm_whole: a GPU per episode

Use case covered. Episodes that need a full GPU each — heavier kernels, or checkers that must not contend for memory — on a multi-GPU instance.

Why this demo. A session is a whole instance, so the native API cannot give one GPU of a multi-GPU box to each session: two ids get two instances, not two GPUs. The Ray farm fixes that — one shared session, one whole GPU per `sub_session`.

Design considerations.

- **Same farm as fractional**, `B4_FARM_GPUS=2`, `B4_GPU_FRACTION=1.0`.
- **Capacity = the GPU count;** `CONC` gating queues overflow in-process.
- **No memory contention, no co-tenant wedge**, at one GPU per concurrent episode.

Measured. Two smokes (2026-09-21): `eval/kernelbench` **0.65** and **0.70**. [PENDING: demo_gpu_farm_whole re-run after the split].

Ch. 7 — GPU isolation on a shared instance

Host side = the CPU pool's (own uid, `0700` dir, `prlimit`, kill-and-scrub). The GPU adds:

Fractional = software isolation on shared hardware; **whole** removes memory contention and the wedge; only **`demo_gpu_single`** is a hardware boundary. Live check (2026-09-25, local farm on GPU 6): slot A and slot B ran as separate uids 20000 / 20001; B was denied A's files, dirs and home; `../`, absolute and symlink escapes were refused; 4 separate CUDA contexts, the 0.5 memory cap held in both slots; closing A left no process and removed its dir. On the AgentCore g6 (no capabilities) the farm falls back to one shared uid..

Layer	Enforced	Still shared
CUDA context	a separate context per actor process	the same physical GPU and driver
GPU memory	`B4_PER_SANDBOX_MEM_FRACTION` — **cooperative**	a process can bypass it
Caches	`SANDBOX_ISOLATE_HOME=1`: own CUDA / compile cache	default: shared warm cache (poisoning risk)
Faults	`nvcc` `flock`, `fraction=0.5`	**no MIG**: a wedge hits every co-tenant
Kernel / host	as the CPU pool	one kernel, one instance

Ch. 8 — demo_cpu_branching (experimental)

Use case covered. A real coding-agent CLI (Claude Code) as the harness; one episode fans out into sub-agents and compacted contexts.

Why this demo. Every other demo maps one episode to one sample; here an episode is a **tree of samples**, and the update must stay unbiased.

Design considerations.

- **Rollout-level GRPO:** advantage and token-mean loss per episode.
- **CLI next to slime; only its tools in the sandbox** (node 22 + `claude`).
- **`MODE=A`, `dp_size=1`** (TP2 × PP2), sync checkpoint save.
- **The CLI needs the tool parser** (`qwen25` / `qwen3`) and **a per-turn cap above Qwen3's `<think>`** (`MAX_RESPONSE_LEN` 2048).

Ch. 8 — demo_cpu_branching: no longer blocked

Three blockers, all fixed:

1. **The CLI's root guard** — fixed with `IS_SANDBOX=1` (2026-09-19).
2. **One-turn episodes, reward 0** (2026-09-19 smoke: exit 0, `fan_out=1`): `run.sh` lacked the SGLang tool / reasoning parsers, so Qwen3's `<tool_call>` came back as a plain `end_turn` text reply and the CLI stopped. Fixed 2026-09-25.
3. **CLI exit 1, "response exceeded the output token maximum"** when a turn hit the 1024 per-turn cap (`stop_reason=max_tokens`). The cap is now `MAX_RESPONSE_LEN`, default 2048.

Verified without a trainer: the session starts, `claude 2.1.278` runs as root, the dial-back to `:18021` works, the parser extracts the tool call, and the checker scores 0 / 0 / 1.

RL result: [PENDING: branching RL smoke fan_out and reward].

Section 6

6. Cross-cutting mechanisms

Episode shape decides what you scale

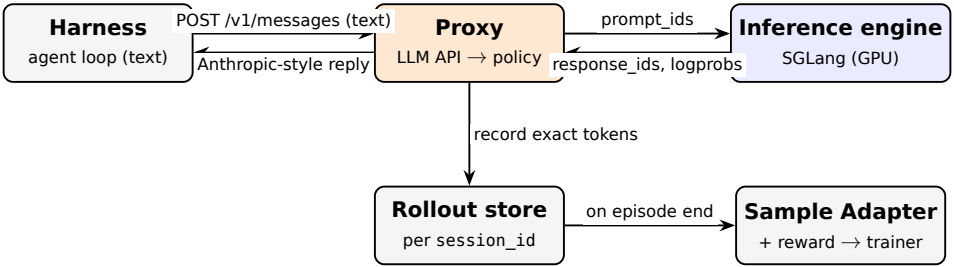
The task's shape — not the domain — sets the config. **One microVM serves many families**: the sandbox is task-agnostic (`write_file` / exec / `read_file`); task specificity is *files seeded per episode*, not a new image.

- **`--rollout-max-response-len`** — per-turn generation budget; set from the family's p95 per-turn response (math 1k–32k; code 100–800; CUDA kernel 200–1k; tool call 50–300).
- **`EPISODE_MAX_TURNS`** (default 3) — the single biggest lever on episode cost: single-turn → 1; iterative code/SQL → 3–5; long-horizon SWE/web → 15–50+.

Regime	Example	Bottleneck	Scale
single-turn, long trace	math, reasoning	**generation**	inference
few-turn, heavy checker	KernelBench (CUDA)	**sandbox**	sandbox / GPU
multi-turn, big observations	SWE, RAG, web	context growth	both + warm session

Mechanism spotlight — TITO & why the proxy exists

A harness works in **text**; RL trains on **token IDs**. Re-tokenizing decoded text can drift from what the model actually sampled — corrupting the action, the importance ratio, and the loss mask.



Token-in-token-out (TITO): the proxy records the **exact sampled `token_ids` + logprobs** at generation time and forwards them intact, so the trainer learns from the tokens that were really sampled.

Mechanism spotlight — TITO & why the proxy exists (cont.)

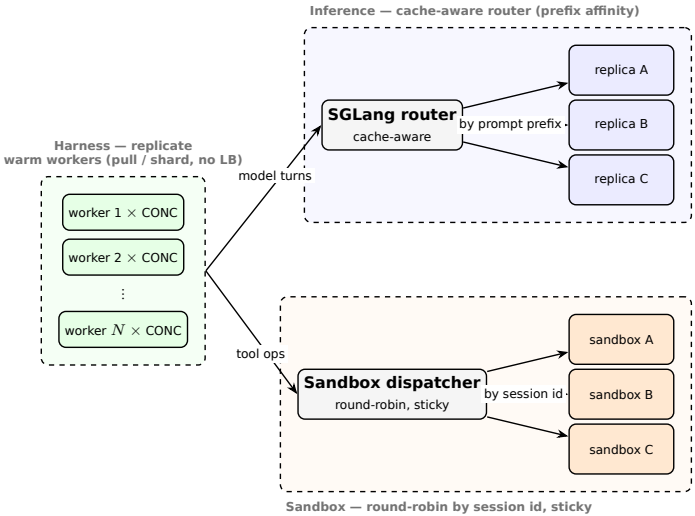
- The proxy is the only component straddling both sides (text ↔ tokens), so it is the only correct place to capture them.
- For binary RLVR rewards this token-level exactness is not optional — slop silently corrupts the signal.

The scaling demo (1/2) — the concrete setup & the three tiers

On the 8×L40S box the demos **fix trainer, inference, and harness** and change only the sandbox ARN + task — the reference-architecture disaggregation made concrete:

Component	Where	Config (shared)
Trainer	4 GPUs	Megatron, GRPO, `global_batch_size=32`, `ACTOR_GPUS=4`
Inference	4 GPUs	SGLang, 2 engines, one **cache-aware router**, `ROLLOUT_GPUS=4`
Harness	in-process (CPU)	`run_episode`, fully-async `MODE=B`, `CONC=4`, `EPISODE_MAX_TURNS=3`
Sandbox	**remote AgentCore**	CPU microVM (arm64) **or** GPU g6 (amd64); one session/episode

The scaling demo (1/2) — the concrete setup & the three tiers (cont.)



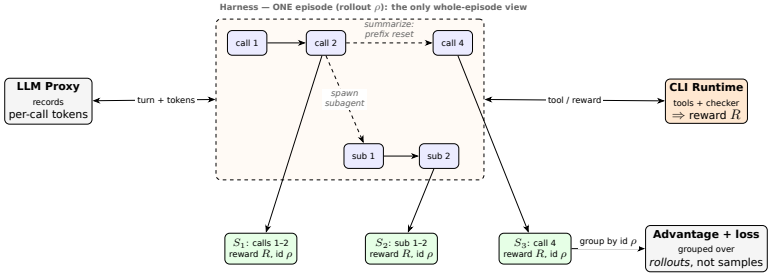
The scaling demo (2/2) — the min() law & `cpu_scale_lanes`

The min() law — `throughput = min(inference, sandbox, harness).` Scale two and the bottleneck moves to the third.

- **Diagnose first:** compare `perf/sglang_gen_ms_*` vs `perf/sandbox_ms_*`. Generation dominates → add SGLang replicas; sandbox dominates → add sessions / ARNs / the GPU farm.
- **`cpu_scale_lanes`** makes all three dials visible as **independent pools** (4 engines : 256 workers : 256 microVMs), including the **easily-forgotten third**: the warm in-process harness (GIL, `to_thread` pool, FDs). Scale it by **replicating warm workers**, not by making it ad-hoc. **Problem targeted:** rollout driver becomes the bottleneck once inference and sandbox are both fast.
- **Load-balancing differs by side:** inference → target slime's cache-aware router (prefix affinity); sandbox → the **session id *is* the balancer** (sticky round-robin across a pool of ARNs). You never stand up your own load balancer.

Mechanism spotlight — one episode → many samples

The reconciliation `cpu_branching` implements, and the reason a **real** agent needs it. A production agent (Claude Code CLI) **fans one episode out into many samples** — it spawns subagents (the Task tool) and summarizes context (resetting the token prefix). Scored naively per-sample, that fan-out silently reweights the GRPO baseline and the loss [1].



Mechanism spotlight — one episode → many samples (cont.)

Solution: the CLI runs **inside a CLI-capable CPU Runtime** (image bakes node22 + `claude`); one episode returns a `list[Sample]`, all tagged with the shared `rollout_id` ρ and the episode's terminal reward R. **Advantage baseline + loss are normalized per rollout, not per sample** (token-mean loss), so an incidental fan-out can't reweight the update. `MODE=A`, `dp=1` (TP2×PP2), sync save.

`demo_cpu_branching` (§5, Ch. 8) implements this; its three blockers are fixed and verified without a trainer — RL result [PENDING: branching RL smoke fan_out and reward].

Mechanism spotlight — resilience: heartbeat & snapshot

Two levers a long RL run composes, demonstrated by `HEARTBEAT=1` and `gpu_single SNAPSHOT_EVERY=1` (§5). **Use case:** any multi-hour run whose warm, stateful sessions (installed deps, seeded repo) must survive both the periodic pause and a genuine instance recycle.

- **Heartbeat (`HEARTBEAT=1`, any demo on an own Runtime; a no-op on the Code Interpreter — Ch. 4)** — during weight sync generation stalls; an idle session is **reclaimed (~15 min)** [D3] and the next round pays a cold start. A `SessionKeeper` pings on a timer, **resetting the idle clock**, so the *same* warm microVM survives. Lives **outside the harness** (`KeptSandbox`), because the stall is a training-loop property, not an episode one.
- **Snapshot (`gpu_single SNAPSHOT_EVERY=1`)** — a recycled GPU instance comes back blank (no VM/EBS snapshot), so the workspace is serialized to a store you own (S3) and restored onto a fresh session.
- **The pairing:** *heartbeat prevents loss; snapshot recovers from it.*

Adoption path & limits — from pilot to production

Day-1 pilot (a few hours):

1. Start on the **default Code Interpreter** (`SANDBOX=agentcore`): no image, no Runtime to deploy.
2. Point slime at it with the `--custom-generate-function-path` hook — **no fork of slime**. Only if the task needs a custom image: build it (arm64), deploy a Runtime (`CreateAgentRuntime` [D9]) and set `SANDBOX=agentcore-runtime`.
3. Seed a task family, run the standard 10-step loop, watch `eval/<task>` and the `[sbx-op]` / `[sglang-gen]` latencies.
4. Flip **CPU↔GPU** with one env var when your checker needs CUDA.

Limits & quotas to plan around:

- **Session length:** 8 h on a CPU microVM, 14 d on a GPU Instance; **idle ~15 min → reclaimed** (use `HEARTBEAT=1` to survive the weight-sync pause).
- **Image size:** ~2 GB arm64 cap — install heavy deps at `CMD`-time, or run it on a warm CPU Instance (`demo_cpu_instance_pool`).

Adoption path & limits — from pilot to production (cont.)

- **GPU sharing: no MIG** on L4 / L40S — cross-session GPU sharing is software-only (the Ray farm), not hardware-partitioned.
- **Next step after training:** serve the resulting policy on Bedrock / SageMaker — the trained checkpoint is a standard model artifact.

Summary

- **Can:** isolated per-session sandbox for untrusted output (CPU default), fully in-account (IAM/SigV4, VPC), long sessions, bring-your-own container, **GPU on Instances**; a one-hook slime integration with the harness in-process.
- **Cannot:** a microVM has no GPU; no cross-session GPU fractioning (no MIG on L4); a session can't exceed 8 h without moving to Instances (14 d).
- **How to run:** default Code Interpreter (or your own Runtime / GPU capacity provider when needed) → seed a task → `invoke_agent_runtime` per tool call → in-sandbox checker gives the reward → slime trains via one `generate` hook. **CPU↔GPU is one env var.**

One image, N sessions, one hook: AgentCore is the isolated per-episode sandbox; slime does the learning.

Section 7

References

References — papers, models & tech reports

- **[1] Agent Lightning** — harnessed agentic RL ·https://arxiv.org/abs/2508.03680
- **[2] DeepSeek-R1** ·https://arxiv.org/abs/2501.12948
- **[3] Qwen3 Technical Report** ·https://arxiv.org/abs/2505.09388
- **[4] Llama-Nemotron** ·https://arxiv.org/abs/2505.00949
- **[5] Kimi K2** ·https://arxiv.org/abs/2507.20534
- **[7] KernelBench** · https://huggingface.co/datasets/ScalingIntelligence/KernelBench
- **[8] MBPP** ·https://arxiv.org/abs/2108.07732
- **[9] slime** — the trainer ·https://github.com/THUDM/slime ·docs https://thudm.github.io/slime/
- **[27] SGLang / RadixAttention** ·https://arxiv.org/abs/2312.07104 ·cache-aware router https://lmsys.org/blog/2024-12-04-sglang-v0-4/

References — RL algorithms & systems (further reading)

- **[13] GRPO (DeepSeekMath)** ·https://arxiv.org/abs/2402.03300
- **[14] DAPO** ·https://arxiv.org/abs/2503.14476
- **[15] GSPO** ·https://arxiv.org/abs/2507.18071
- **[18] AReaL** — asynchronous RL system ·https://arxiv.org/abs/2505.24298
- **[19] HybridFlow / veRL** ·https://arxiv.org/abs/2409.19256
- **[22] Truncated Importance Sampling** — train/inference mismatch · https://fengyao.notion.site/off-policy-rl
- **[28] APRIL** — active partial rollouts ·https://arxiv.org/abs/2509.18521

References — AgentCore docs, SDK & CLI

- **[D2] Host with Runtime** — isolation, compute types · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agents-tools-runtime.html
- **[D3] Runtime — how it works** — idle 15 m / max 8 h · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html
- **[D5] Instances (GPU)** ·https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-instances-how-it-works.html
- **[D8] InvokeAgentRuntime** ·https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html
- **[D9] CreateAgentRuntime** ·https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_CreateAgentRuntime.html
- **[D13] Observability** — vCPU / GB-hour metrics ·https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html
- **[D15] Python SDK** — `@app.entrypoint` · https://github.com/aws/bedrock-agentcore-sdk-python

References — AgentCore docs, SDK & CLI (cont.)

- **[D16] AgentCore CLI** — `@aws/agentcore` · https://www.npmjs.com/package/@aws/agentcore
- **[D17] Runtime service contract** — :8080, /invocations, /ping · https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-service-contract.html